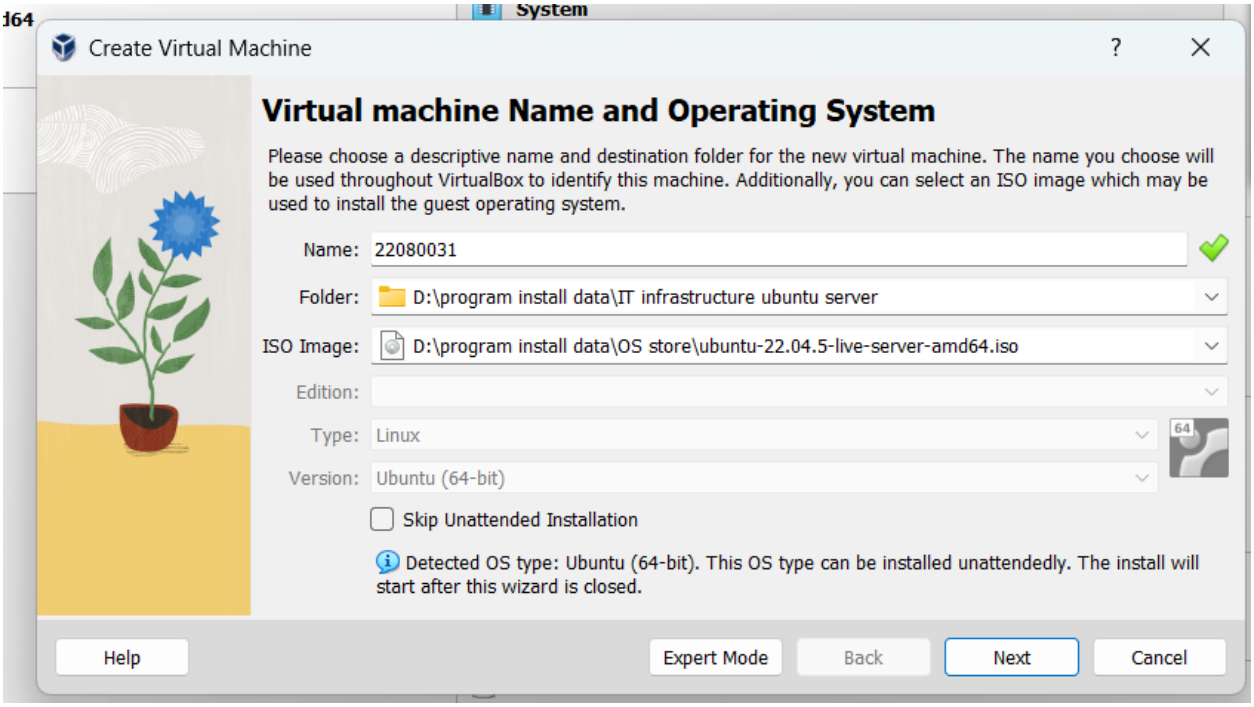
IT Infrastructure Task 2 Ubuntu Server DHCP and DNS

This report documents the Ubuntu Server virtual machine, DHCP address assignment, and DNS resolution tests.

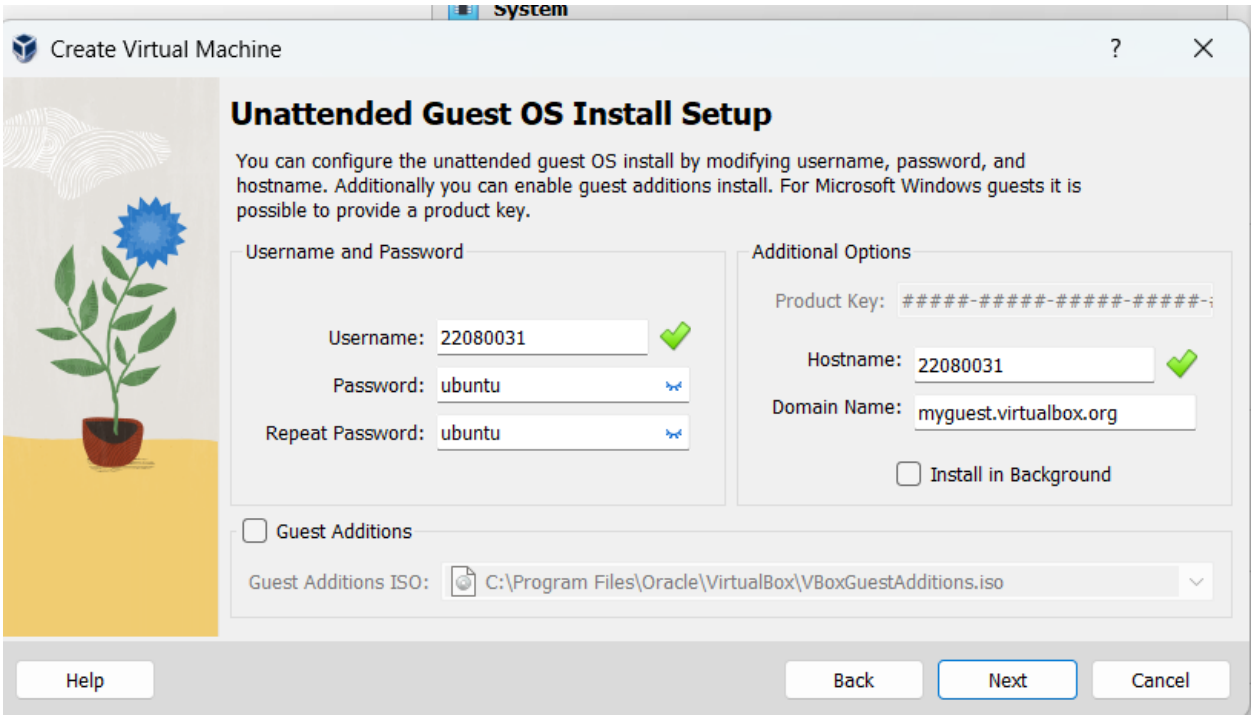
Building and configuring ubuntu server

Firstly, set the machine name of my student ID and choose the ubuntu server ISO disk



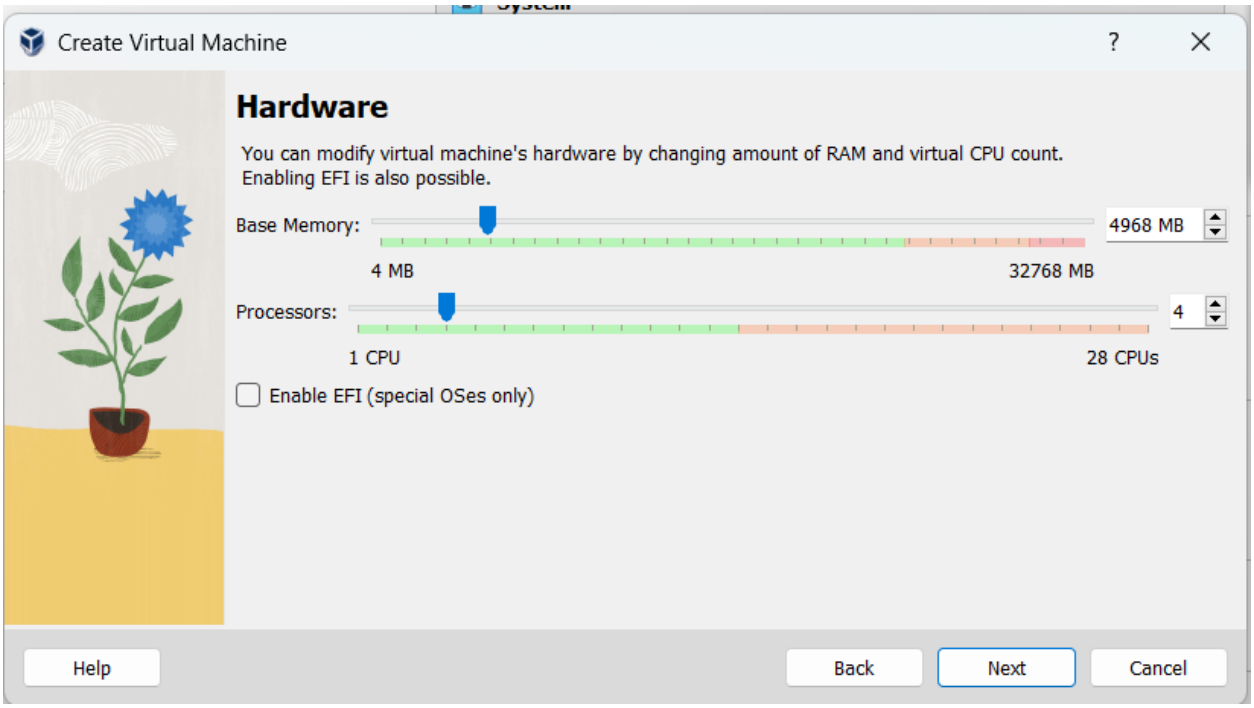
The VirtualBox wizard identifies the new Ubuntu Server virtual machine and points it to the Ubuntu 22.04.5 installation ISO. The VM name distinguishes this lab server from the client machine used later.

Setting password and username



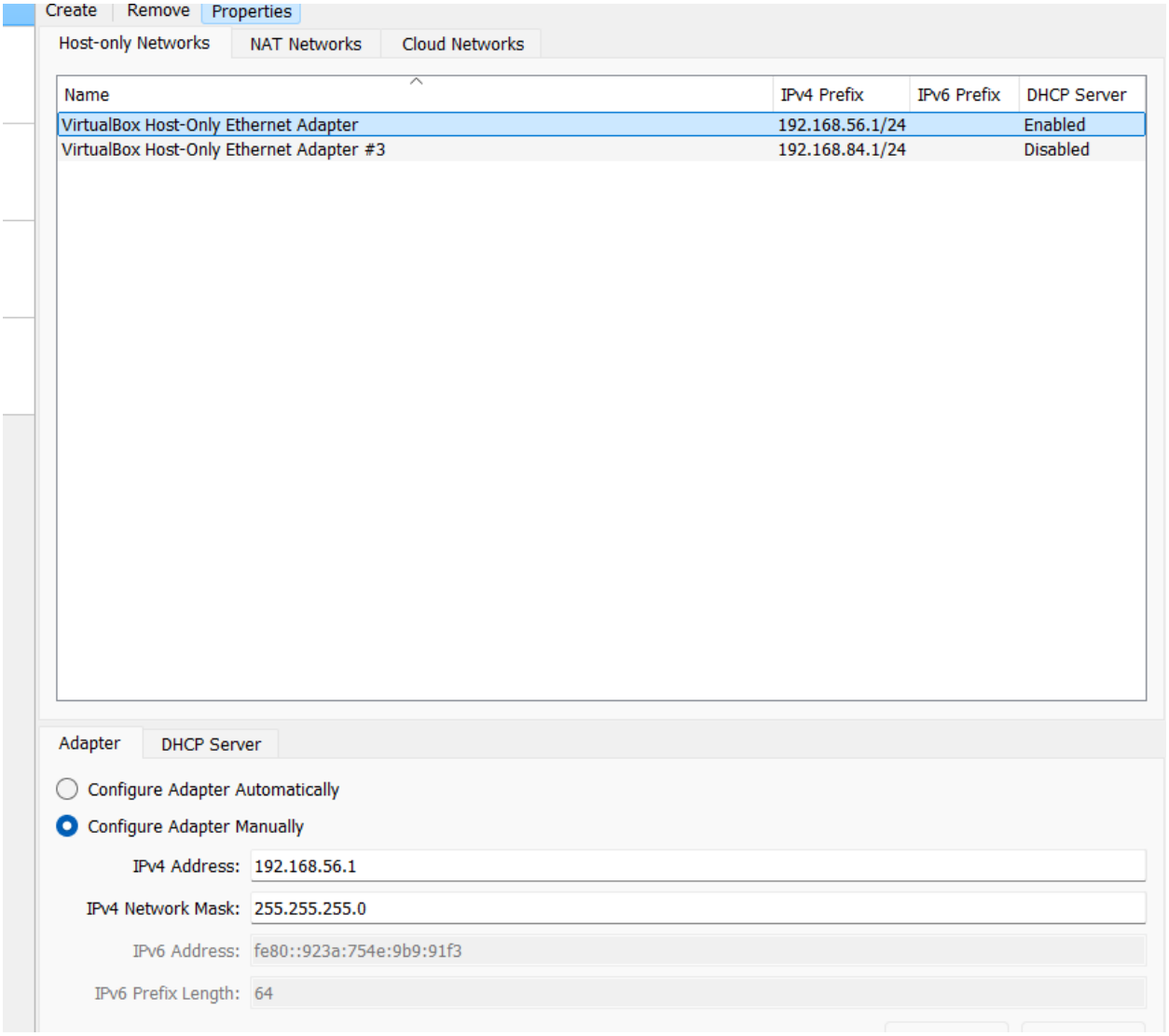
The unattended-installation screen assigns the initial account and host name. These credentials are for the lab VM; a real server would need a unique strong password kept out of screenshots.

Set up the hardware

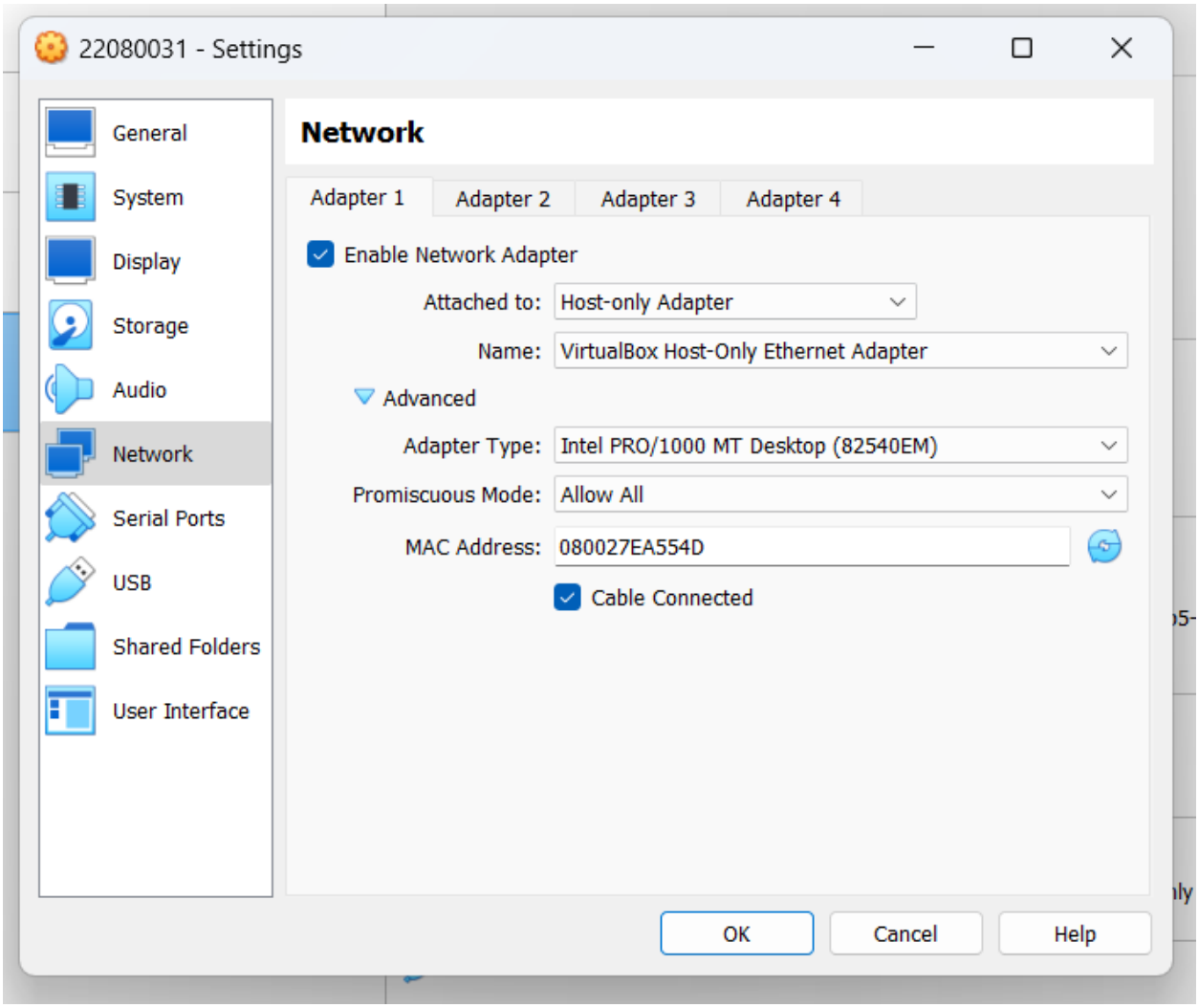


The hardware page allocates about 5 GB of memory and four virtual CPUs. This gives the guest enough resources for the later DHCP and DNS services while leaving resources for the host.

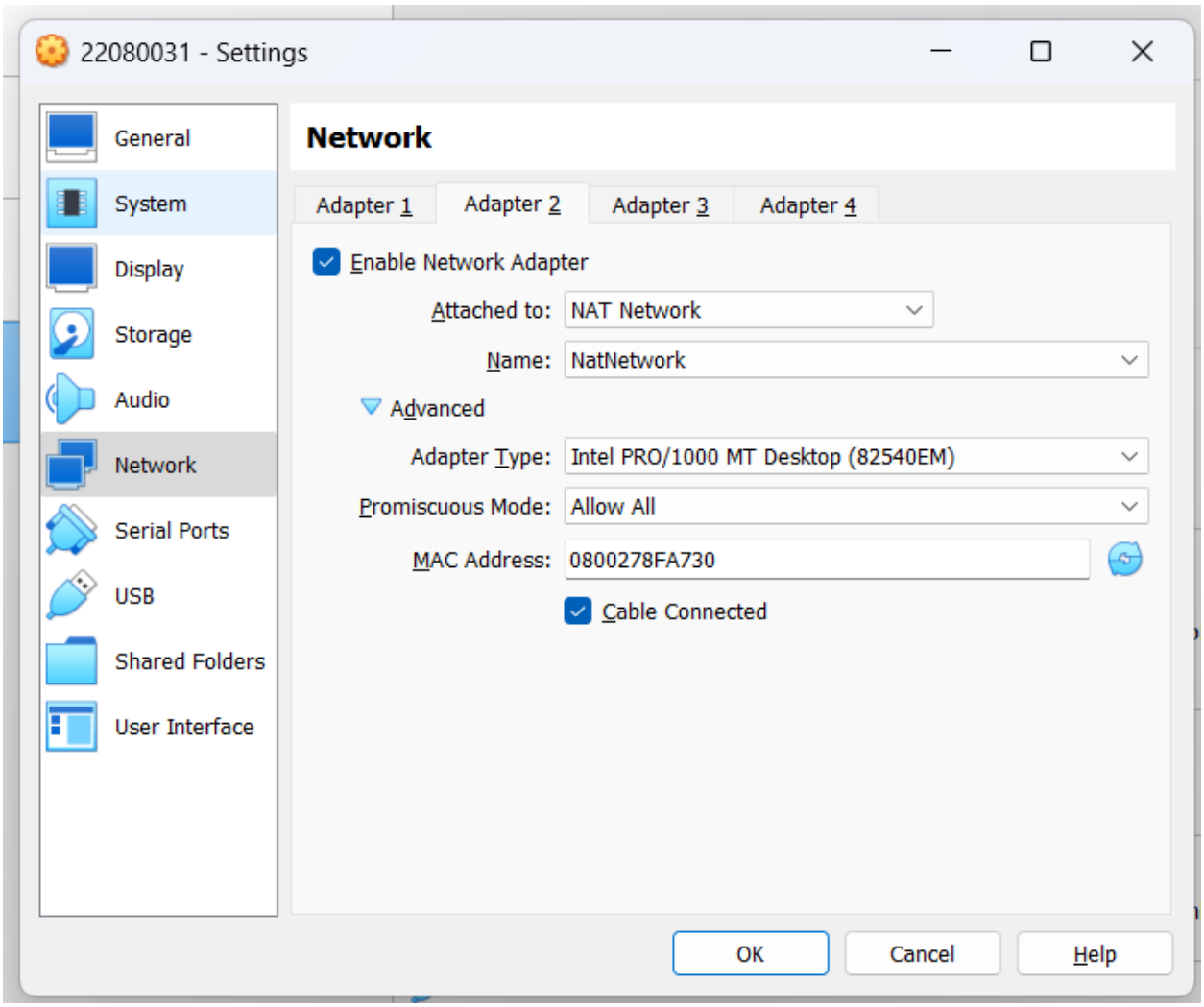
Configure the host-only adapter and NAT network in the virtual box. The host-only adapter creates a private network between our host computer and the guest VMs and NAT network to allow your guest VMs to access external networks.



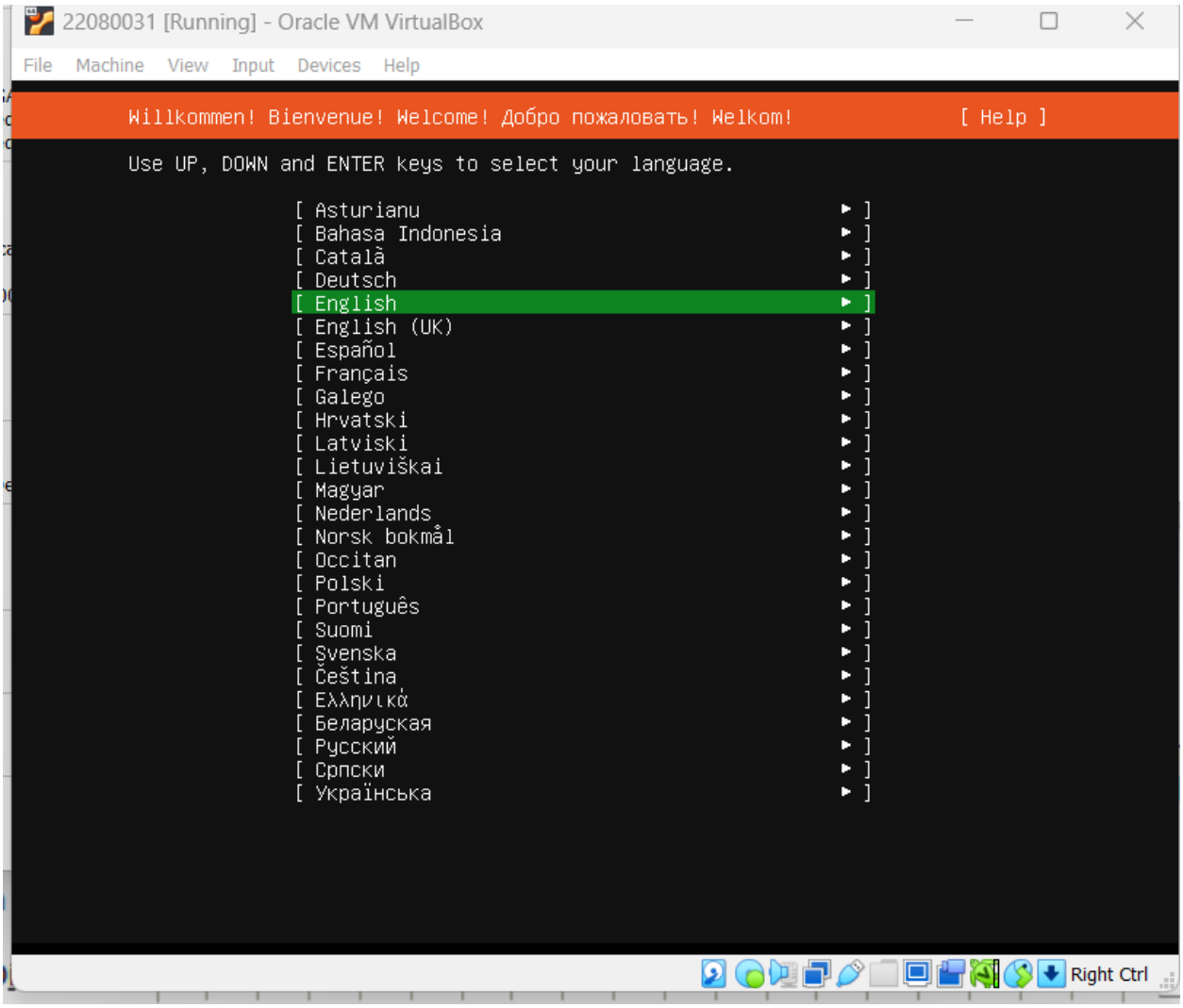
The host-only network is configured as 192.168.56.0/24, with the host adapter at 192.168.56.1. It provides a private path between the host and guest VMs for the service tests.



Adapter 1 connects the Ubuntu server to that host-only network. The selected adapter is the one that later receives the 192.168.56.118 address and serves lab clients.

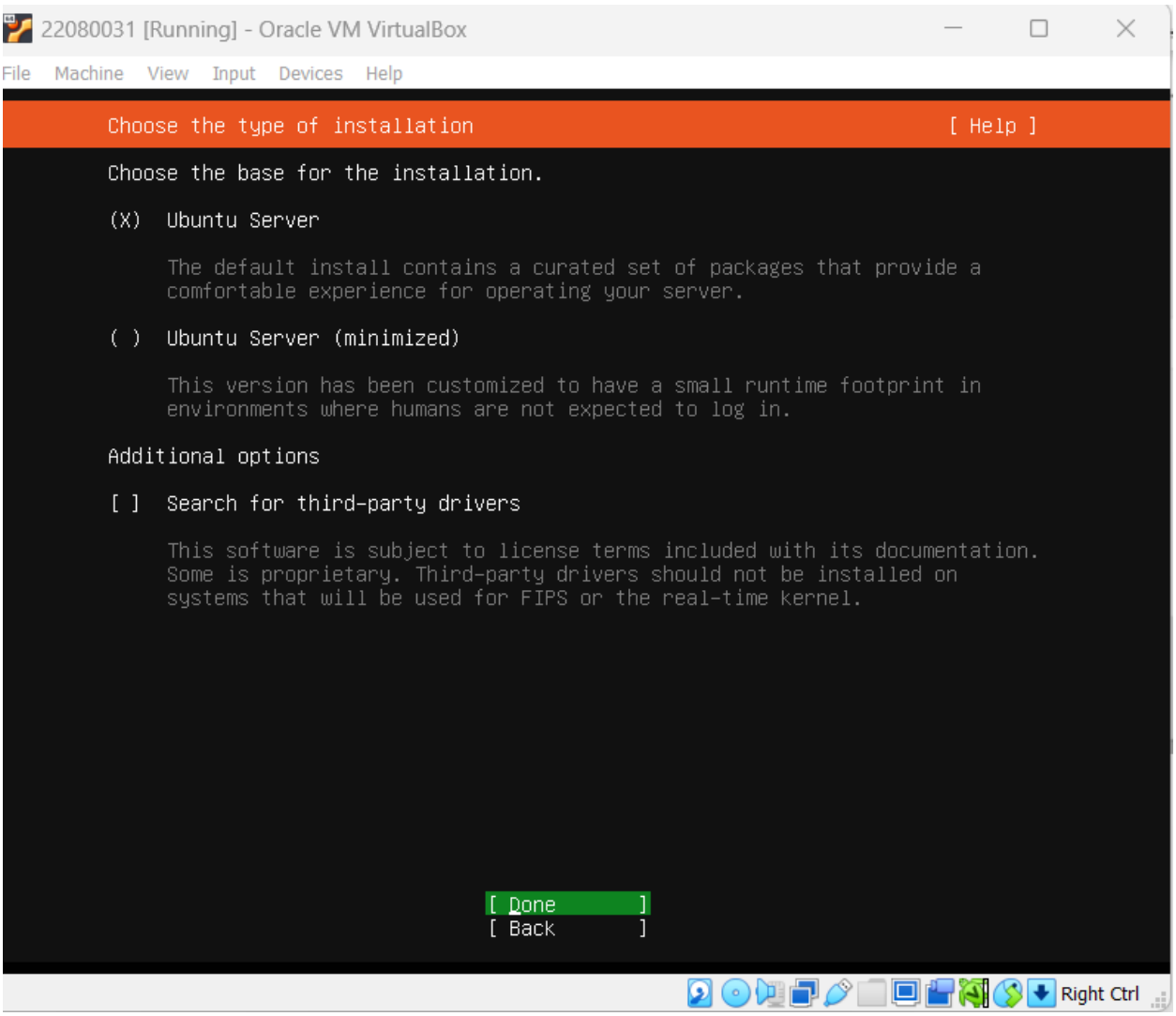


Adapter 2 uses a NAT Network. This separate connection gives the VM an outbound route for package downloads without placing the lab services directly on the external network.



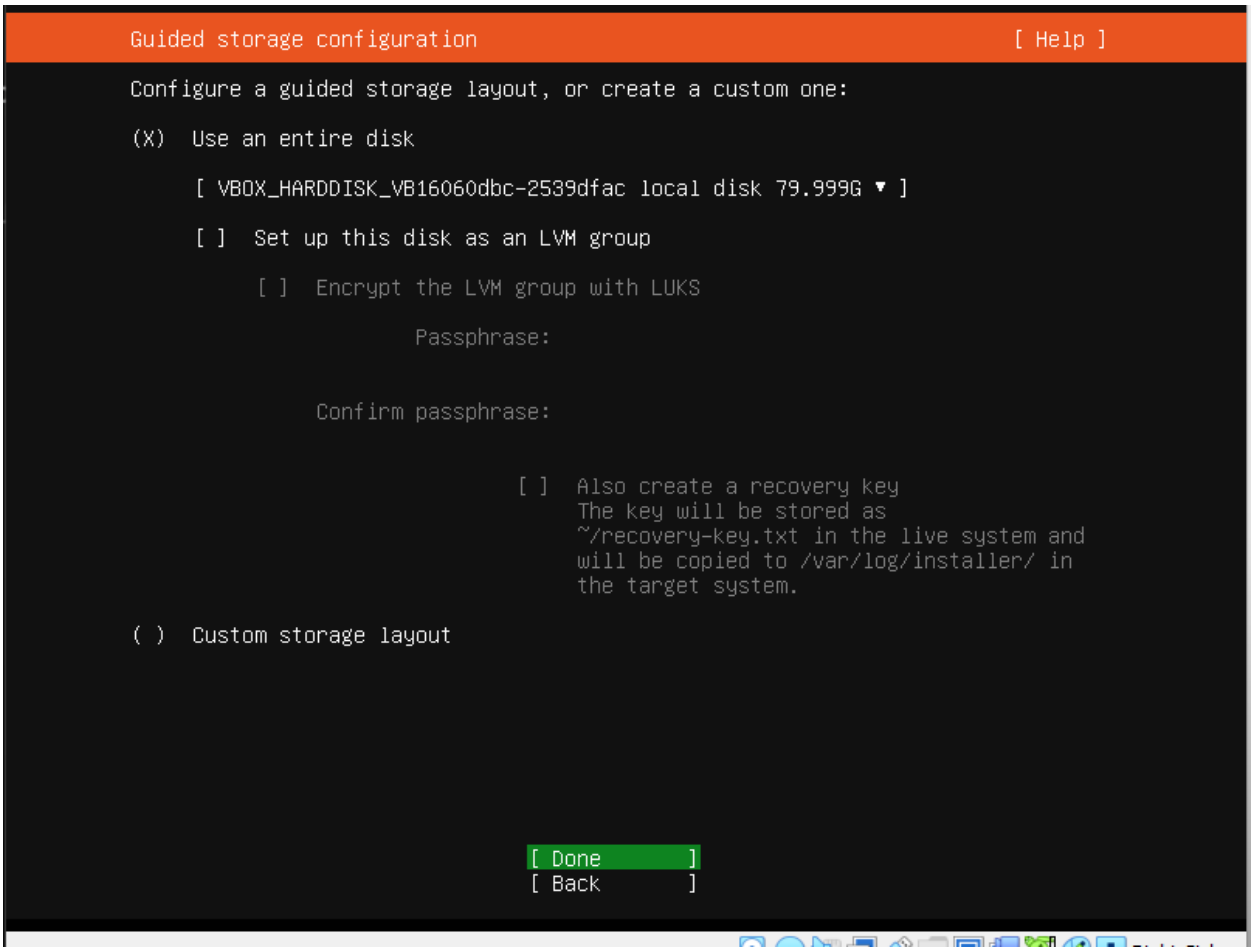
The installer starts with English selected. This choice determines the language of the remaining setup screens and does not affect the network configuration.

Select ubuntu default installation for experience full functionality:



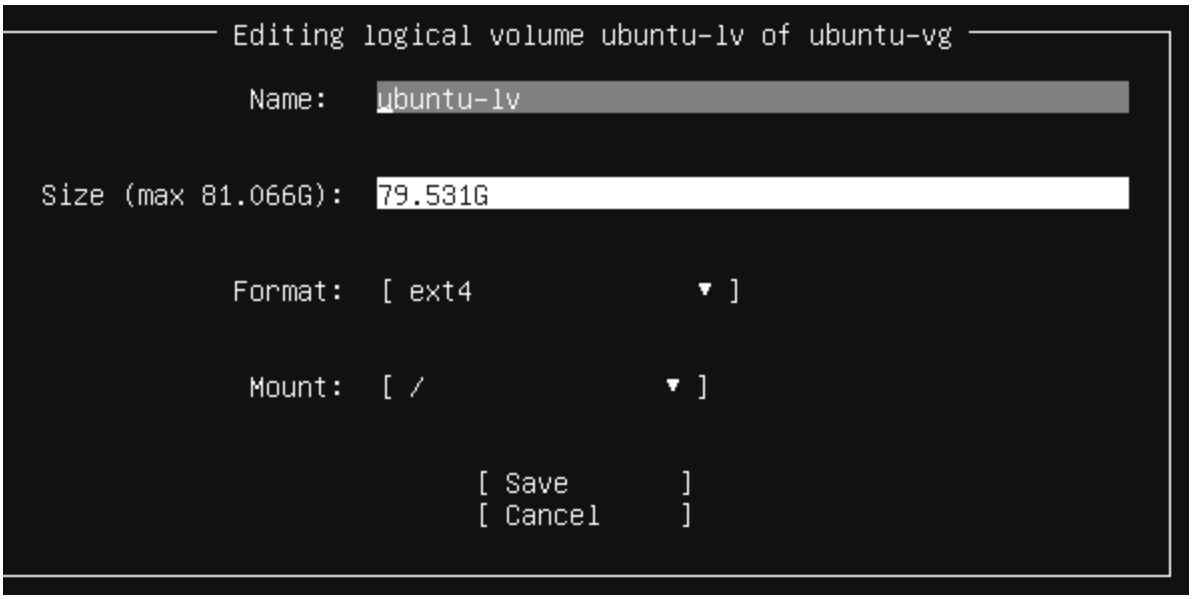
The standard Ubuntu Server installation is selected rather than the minimized option. It provides the normal set of server utilities used in the following tasks.

Choose to use an entire disk:

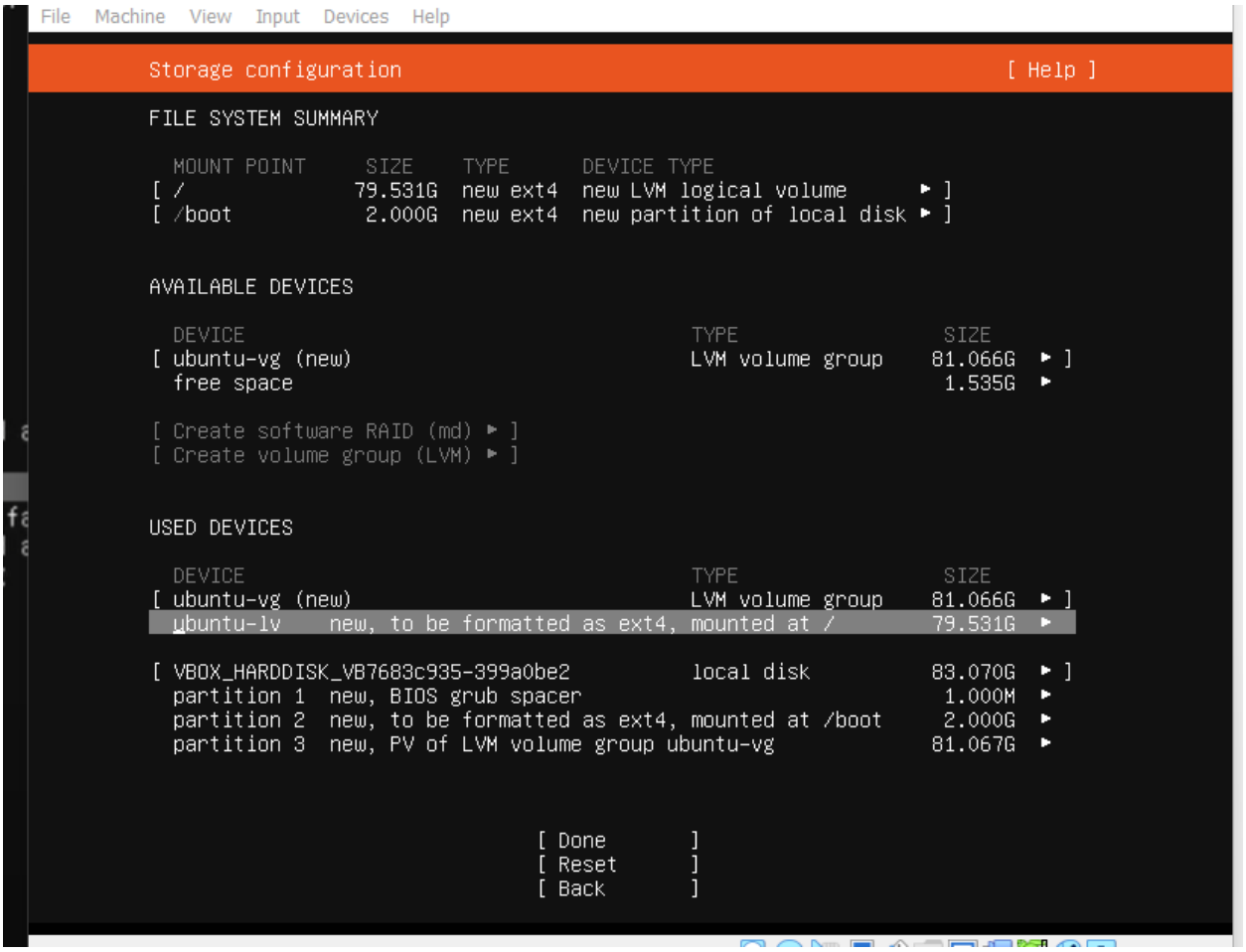


The guided storage screen selects the virtual disk for installation. The choice applies only to the VM's virtual disk, which the installer will partition and format.

In the default setting, ubuntu server has a lot of free space at the start. Therefore, give space to our ubuntu file system by adjusting the disk space:

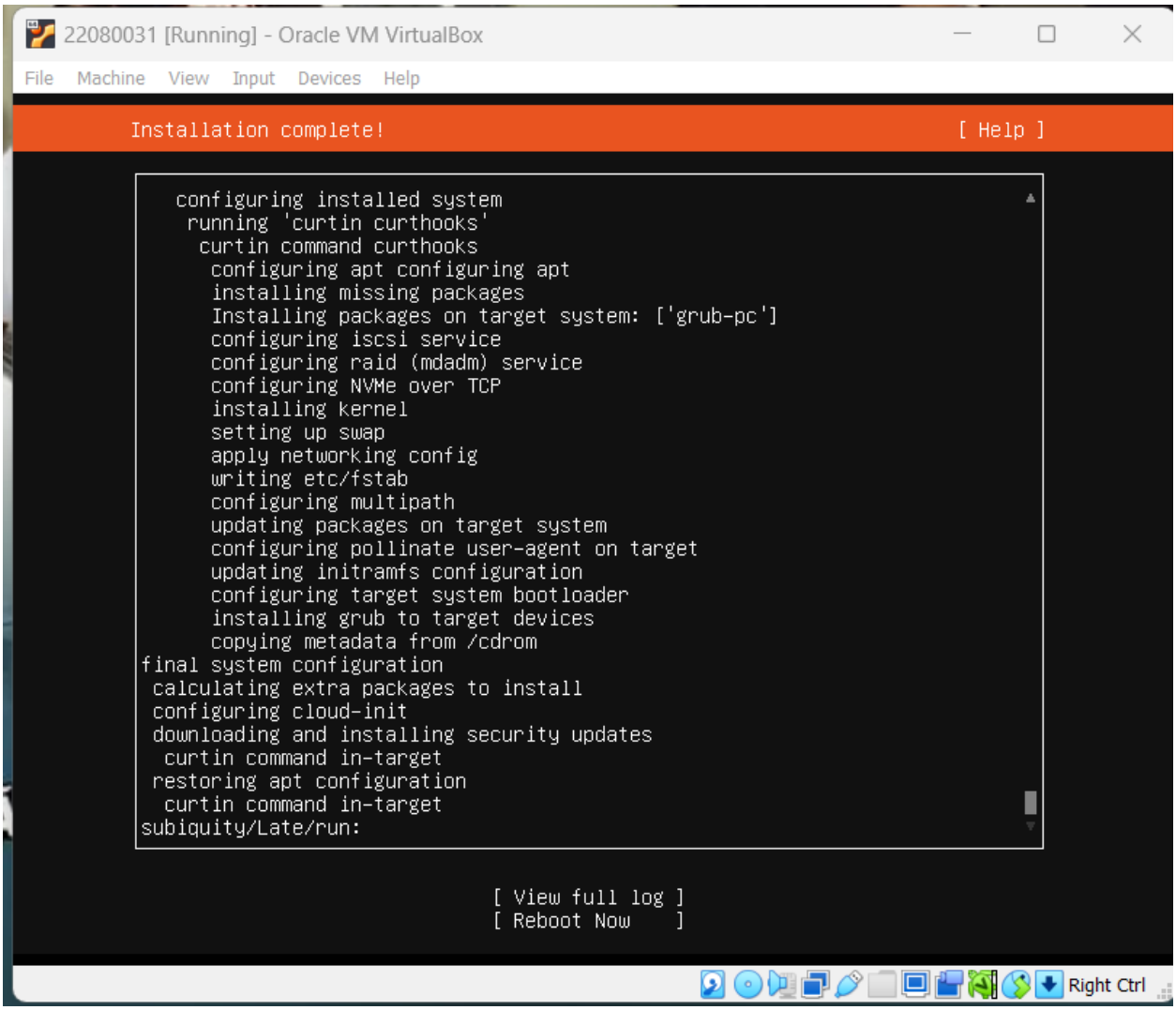


The root logical volume is expanded to about 79.5 GB and formatted as ext4. Enlarging the root filesystem makes the virtual disk space available to the installed operating system.

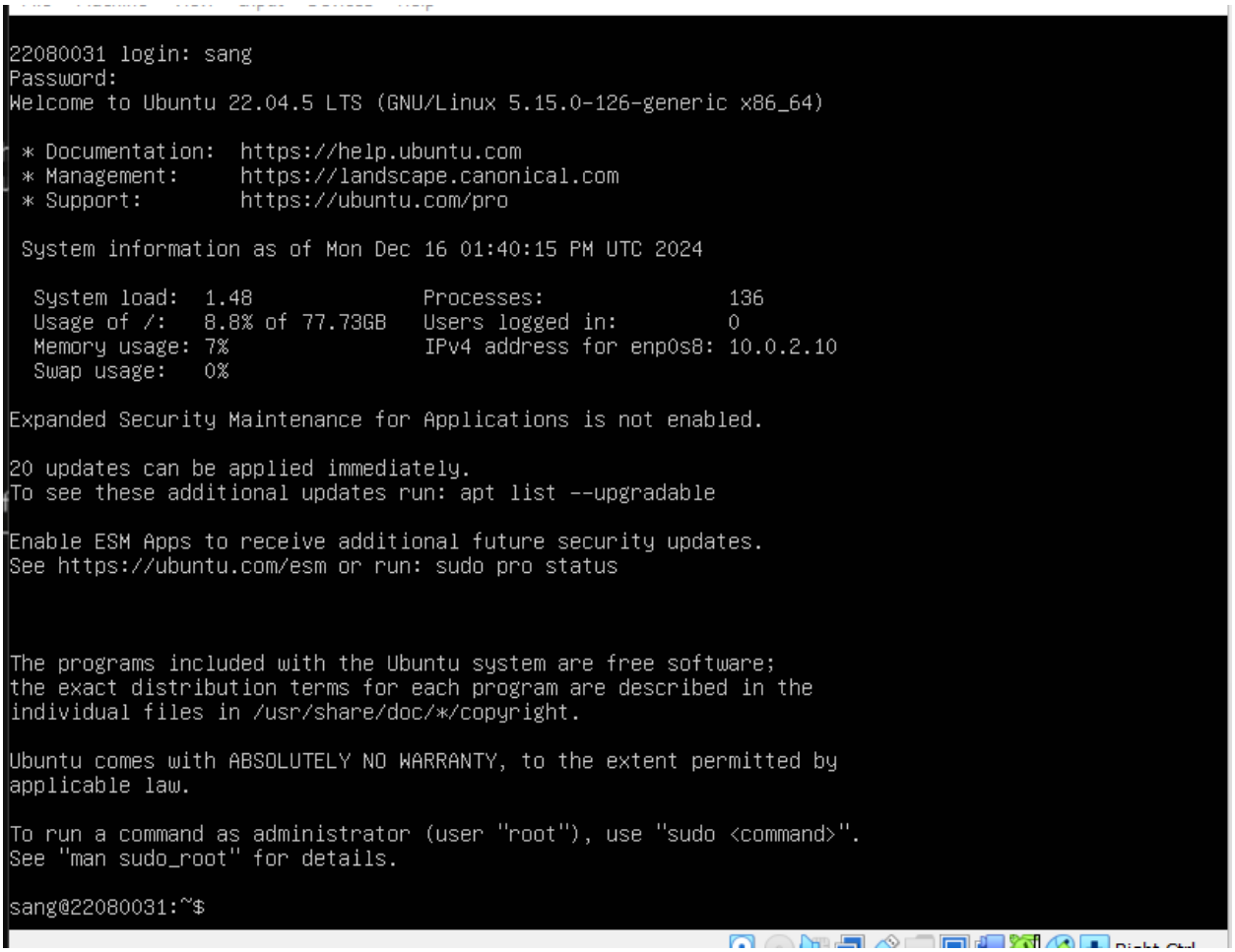


The storage summary confirms the root and boot filesystems before installation. This check matters because accepting the layout writes the partitions to the virtual disk.

After that, reboot the server

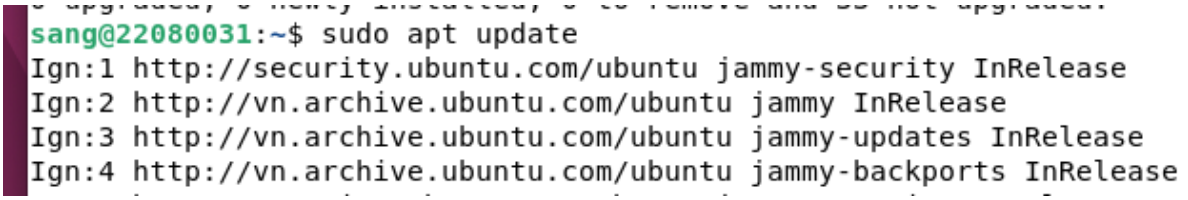


The installer reports that system configuration has completed and offers a reboot. Rebooting starts the newly installed operating system from its virtual disk.



The login screen shows that Ubuntu Server boots and accepts the configured account. It is the starting point for the service installation commands that follow.

Configure DHCP



The package index refresh is attempted before installing DHCP. Updating package lists helps the package manager locate current software versions; any mirror errors should be resolved if installation cannot continue.

Task 1: Download isc-dhcp-server

```
sang@22080031:~$ sudo apt-get install isc-dhcp-server
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
isc-dhcp-server is already the newest version (4.4.1-2.3ubuntu2.4).
0 upgraded, 0 newly installed, 0 to remove and 33 not upgraded.
```

The installation command checks for isc-dhcp-server and reports that the package is already present. This package supplies the DHCP daemon that will lease addresses to clients.

```
sang@22080031:~$ sudo ifconfig
[sudo] password for sang:
Sorry, try again.
[sudo] password for sang:
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.56.118  netmask 255.255.255.0  broadcast 192.168.56.255
        inet6 fe80::a00:27ff:fe50:98a6  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:50:98:a6  txqueuelen 1000  (Ethernet)
        RX packets 2  bytes 1180 (1.1 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 55  bytes 8078 (8.0 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.12  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::a00:27ff:fe78:5a0f  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:78:5a:0f  txqueuelen 1000  (Ethernet)
        RX packets 11  bytes 4292 (4.2 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 64  bytes 8856 (8.8 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
```

The interface output shows 192.168.56.118 on the host-only side and 10.0.2.12 on the NAT side. The DHCP service must use the 192.168.56.x interface so that it reaches the lab client.

Task 2: Configure the DHPC Server.

In /etc/default/isc-dhcp-server, configure the enp0s3 as the interface that our DHCP server should listen for incoming DHCP requests:

```
sang@22080031:/etc/dhcp$ sudo nano /etc/default/isc-dhcp-server
```

The default DHCP service file is opened for editing. This file selects which network interface accepts incoming DHCP requests.

```
INTERFACESv4="enp0s3"
INTERFACESv6=""
```

INTERFACESv4 is set to enp0s3 and the IPv6 field is left empty. That confines the IPv4 DHCP service to the host-only network shown in the earlier interface output.

Next, configure DHCP settings in /etc/dhcp/dhcp.conf:

```
sang@22080031:/etc/dhcp$ sudo nano /etc/dhcp/dhcpd.conf
sang@22080031:/etc/dhcp$
```

The main dhcpd.conf file is opened to define lease behavior and the address pool. The daemon reads this file when it starts or restarts.

```
  GNU nano 6.2                    /etc/dhcp/dhcpd.conf *
# configuration file instead of this file.
#

# option definitions common to all supported networks...
# option domain-name-servers 192.168.56.102;
default-lease-time 600;
max-lease-time 7200;

# The ddns-updates-style parameter controls whether or not the server will
# attempt to do a DNS update when a lease is confirmed. We default to the
# behavior of the version 2 packages ('none', since DHCP v2 didn't
# have support for DDNS.)
ddns-update-style none;

# If this DHCP server is the official DHCP server for the local
# network, the authoritative directive should be uncommented.
authoritative;

# Use this to send dhcp log messages to a different log file (you also
# have to hack syslog.conf to complete the redirection).
```

The global settings give clients a 600-second default lease and a 7200-second maximum lease. Dynamic DNS updates are disabled, and authoritative mode marks this server as the DHCP authority for the lab subnet.

DHCP setting

Configuration	Description
default-lease-time 600;	Sets the default lease duration to 600 seconds (10 minutes) for clients not requesting a specific time.
max-lease-time 7200;	Limits the maximum lease duration to 7200 seconds (2 hours) to prevent excessively long leases.
ddns-update-style none;	Disables Dynamic DNS updates, meaning the DHCP server won't modify DNS records when assigning IP addresses.
authoritative ;	Designates this DHCP server as the primary authority for its subnet, allowing it to promptly correct clients with incorrect IP configurations.

The accompanying table explains the global lease and authority settings. A short lease is convenient for a lab because clients can obtain revised settings quickly after changes.

```
subnet 192.168.56.0 netmask 255.255.255.0 {
  range 192.168.56.110 192.168.56.130;
  option domain-name-servers ns1.22080031.example.com;
  option domain-name "22080031.example.com";
  option subnet-mask 255.255.255.0;
  option routers 192.168.56.255;
  option broadcast-address 192.168.56.255;
  default-lease-time 600;
  max-lease-time 7200;
}
```

The subnet block offers addresses from 192.168.56.110 through 192.168.56.130 and supplies DNS and domain options. The router option is set to 192.168.56.255 in the screenshot, but that is the /24 broadcast address, not a usable gateway; it needs a valid router address if clients require routing.

DHCP setting

Configuration	Description
subnet 192.168.56.0 netmask 255.255.255.0	Defines the subnet and netmask for the DHCP configuration.
range 192.168.56.110 192.168.56.130	Specifies the range of IP addresses that can be dynamically assigned to clients.
option domain-name-servers ns1.22080031.example.com	Sets the DNS server for clients.
option domain-name "22080031.example.com"	Defines the domain name for clients.
option subnet-mask 255.255.255.0	Specifies the subnet mask for the network.
option routers 192.168.56.255	Sets the default gateway (router) address for the subnet.
option broadcast-address 192.168.56.255	Defines the broadcast address for the subnet.
default-lease-time 600	Specifies the default lease time (in seconds) for DHCP clients (10 minutes).
max-lease-time 7200	Defines the maximum lease time (in seconds) for DHCP clients (2 hours).

The subnet table translates the DHCP directives into client-facing settings. The address range defines the pool, while the DNS, subnet mask, and router options are delivered with each lease; the router value noted above should be corrected.

Task 3: Enable DHCP and Firewall

Start DHCP service by the above commands and verify by checking the status whether service is active:

```
sang@22080031:/etc/dhcp$ sudo systemctl start isc-dhcp-server
sang@22080031:/etc/dhcp$ sudo systemctl enable isc-dhcp-server
Synchronizing state of isc-dhcp-server.service with SysV service script with /li
b/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable isc-dhcp-server

sang@22080031:/etc/dhcp$
```

The service is started and enabled with systemctl. Starting runs it now, while enabling asks Ubuntu to launch it automatically at boot.

```
sang@22080031:/etc/dhcp$ sudo systemctl status isc-dhcp-server
● isc-dhcp-server.service - ISC DHCP IPv4 server
     Loaded: loaded (/lib/systemd/system/isc-dhcp-server.service; enabled; vend>
     Active: active (running) since Wed 2025-03-05 06:24:25 UTC; 31min ago
       Docs: man:dhcpd(8)
   Main PID: 1033 (dhcpd)
      Tasks: 4 (limit: 4555)
     Memory: 5.6M
        CPU: 24ms
     CGroup: /system.slice/isc-dhcp-server.service
             └─1033 dhcpd -user dhcpd -group dhcpd -f -4 -pf /run/dhcp-server/d>
```

The status output shows isc-dhcp-server as active and running. This confirms the process started, though a client lease is still needed to verify end-to-end DHCP behavior.

Check the netstate of DHCP Server to check the running port:

```
sang@22080031:~$ netstat -tulnp | grep dhcp
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
sang@22080031:~$ sudo netstat -tulnp | grep dhcp
udp        0      0 0.0.0.0:67              0.0.0.0:*
1018/dhcpd
```

The socket check shows dhcpd listening on UDP port 67. That is the server-side DHCP port used to receive client requests.

Enable the ubuntu firewall then allow port 67 and 68 on the interface using DHCP

```
sang@22080031:~$ sudo ufw enable
Firewall is active and enabled on system startup
```

UFW is enabled on the server. Turning on a firewall restricts traffic according to its rules, so required DHCP traffic must be allowed before testing clients.

```
sang@22080031:/etc/dhcp$ sudo ufw allow in on enp0s3 from any port 68 to any por
t 67
Rule added
Rule added (v6)
```

The displayed UFW command adds an incoming UDP 67 rule on enp0s3 for requests originating from client port 68. This permits DHCP requests on the lab interface.

Check the firewall status

```
sang@22080031:/etc/dhcp$ sudo ufw status
Status: active

To                         Action      From
--                         ------      ----
67/udp                     ALLOW       Anywhere
67/udp on enp0s3           ALLOW       68/udp
Bind9                      ALLOW       Anywhere
67/udp (v6)                ALLOW       Anywhere (v6)
67/udp (v6) on enp0s3      ALLOW       68/udp (v6)
Bind9 (v6)                 ALLOW       Anywhere (v6)
```

The firewall status confirms that UFW is active and lists the DHCP rules. The screenshot also shows a BIND rule, which belongs to the later DNS portion of the lab.

Task 4: Set up Ubuntu client

Now go to the guest machine and enable the DHCP request by configuring the /etc/network/interfaces with the following commands:

```
netconfig           network/            networks
netplan/            networkd-dispatcher/ newt/
sang@sang-VirtualBox:~/Desktop$ sudo nano /etc/network/interfaces
```

The client opens /etc/network/interfaces to configure its lab adapter. That file is used here to request an address automatically rather than assigning one by hand.

```
  GNU nano 7.2                    /et
auto lo
iface lo inet loopback
auto enp0s3
iface enp0s3 inet dhcp
```

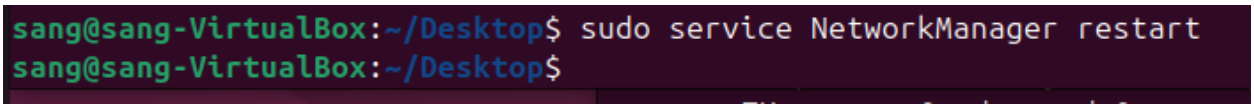
The interface stanza brings up enp0s3 at startup and sets it to use DHCP. On a system managed by NetworkManager or Netplan, the active network manager must also be considered for this setting to take effect.

Network Interface Configuration Table

Configuration	Description
auto lo	Specifies that the loopback interface should be automatically enabled at boot.
iface lo inet loopback	Configures the `lo` interface with the loopback address used for internal system communication.
auto enp0s3	Indicates that the `enp0s3` network interface should be automatically brought up during system startup.
iface enp0s3 inet dhcp	Sets the `enp0s3` interface to obtain its IP address dynamically via DHCP.

The configuration table explains the loopback and client-interface directives. The loopback entry keeps local host communication available, and the enp0s3 entry requests a DHCP lease.

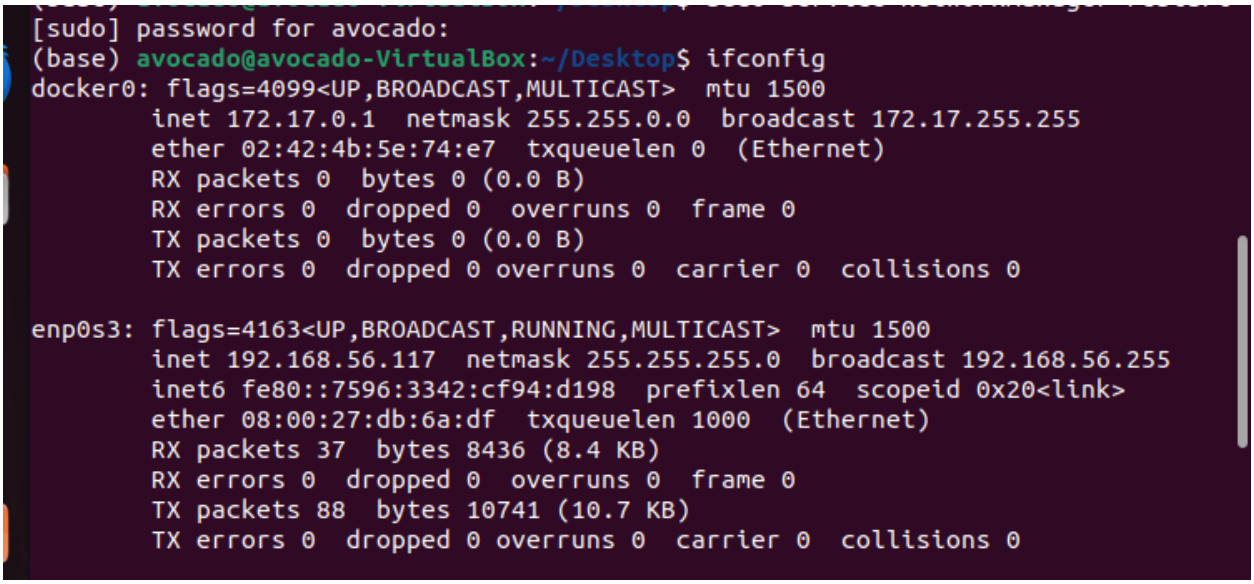
Then, restart the network service



NetworkManager is restarted after the client setting is changed. Restarting the network service makes the client renegotiate its connection and obtain current DHCP options.

Task 5: Test the DCHP connectivity

Ubuntu client network information:



The client interface output shows a 192.168.56.117 address on enp0s3. Because it lies inside the configured pool, it supports that the client received a lease from the lab network.

Server DHCP lease list:

```
sang@demo:~$ sudo dhcp-lease-list
To get manufacturer names please download http://standards.ieee.org/regauth/oui/
oui.txt to /usr/local/etc/oui.txt
Reading leases from /var/lib/dhcp/dhcpd.leases
MAC                IP              hostname       valid until         manufactur
er
================================================================================
===============
08:00:27:50:98:a6  192.168.56.118  -NA-           2025-03-08 14:03:19 -NA-

08:00:27:db:6a:df  192.168.56.117  avocado-Virtua 2025-03-08 14:08:51 -NA-
```

(the hostname is different in this image is because I came back after setting up mail server and redo all the steps)

The server lease list records the client's MAC address and 192.168.56.117 lease. The hostname differs because the lab was repeated later; the matching address and client entry are the relevant evidence.

The MAC, IP, Hostname and Validity date of the Ubuntu client appears in the server's DHCP lease list, so the configurations are done successfully.

DNS

Task 1: Install DNS Server

Install bind9 to run DNS service

```
sang@22080031:~$ sudo apt install bind9 -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bind9 is already the newest version (1:9.18.30-0ubuntu0.22.04.2).
0 upgraded, 0 newly installed, 0 to remove and 33 not upgraded.
sang@22080031:~$
```

The BIND9 installation command reports that the DNS package is available. BIND provides the named service used for forward and reverse name resolution.

Confirm the installation

```
sang@22080031:~$ sudo named -v
[sudo] password for sang:
BIND 9.18.30-0ubuntu0.22.04.2-Ubuntu (Extended Support Version) <id:>
sang@22080031:~$
```

The named version command confirms that the BIND executable is installed. It verifies the software version, not yet that the DNS service is serving the new zone.

Task 2: Configure the DNS Server

Set up the **/etc/hosts** file to translate the hostnames (22080031.example.com) to IP addresses (192.168.56.118):

```
sang@22080031:~$ sudo nano /etc/hosts
```

The server opens /etc/hosts to map its local name before editing DNS. This local file affects resolution on the server itself and is separate from the records BIND will serve to clients.

```
  GNU nano 6.2                          /etc/hosts
127.0.0.1 localhost
127.0.1.1 22080031.example.com  22080031
192.168.56.118 22080031.example.com 22080031

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

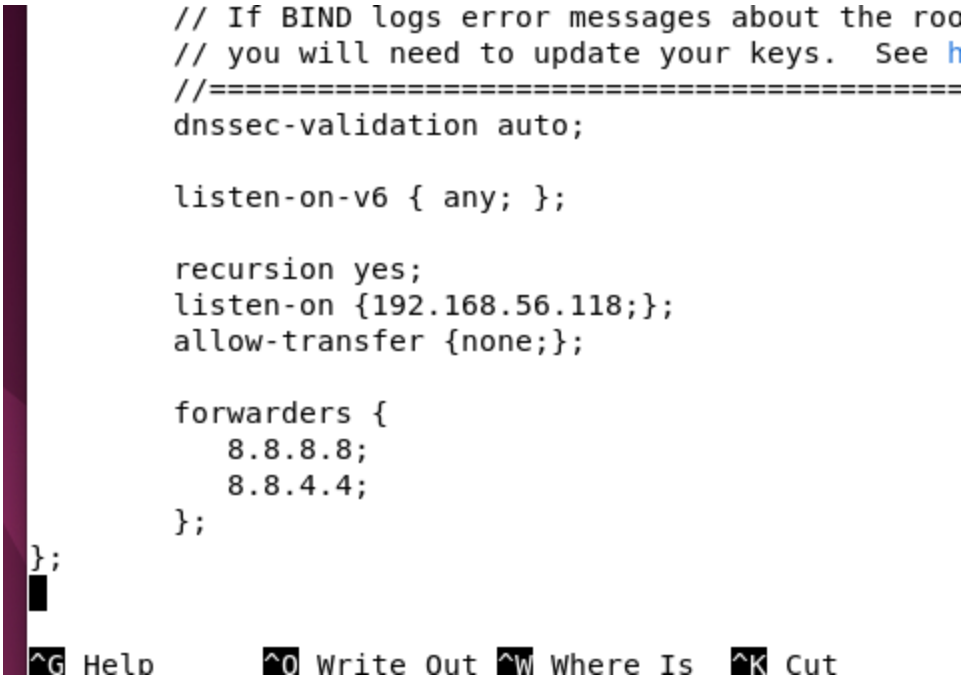
The hosts entry associates 22080031.example.com with 192.168.56.118. It allows the server to resolve that name locally while the DNS zone is being built.

Then in "/etc/bind/named.conf.options", set default behaviors and parameters for the DNS server's operation:

```
sang@22080031:~$ sudo nano /etc/bind/named.conf.options
```

The named.conf.options file is opened for the server-wide DNS settings. These options control recursion, listening interfaces, forwarding, and zone-transfer behavior.

Edit the recursion, listening IP address, allow transfer and forwarders IP



The configuration enables recursive queries, listens on 192.168.56.118, disallows zone transfers, and forwards unresolved names to two external resolvers. Listening on the lab address makes the service reachable by the client.

DNS server setting

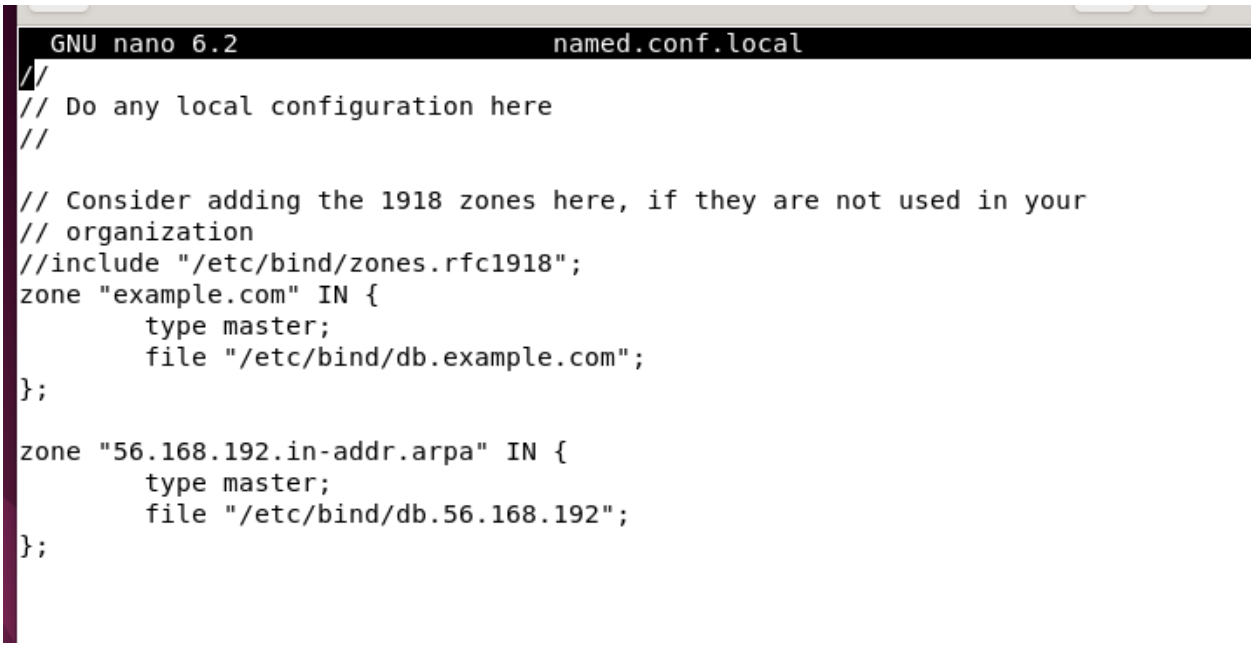
Configuration	Description
Directory "/var/cache/bind";	This sets the working directory for BIND9. It's where BIND stores its cache files, temporary zone files, and other runtime data. Keeping cache files in a dedicated directory helps with organization and ensures that BIND has proper permissions to read and write these files.
dnssec-validation auto;	With this option, BIND9 is configured to automatically validate DNSSEC-signed responses. DNSSEC (DNS Security Extensions) adds a layer of trust by verifying the integrity of DNS data. The "auto" setting enables BIND to fetch the necessary trust anchors without manual intervention.
listen-on-v6 {any;} ;	Disables Dynamic DNS updates, meaning the DHCP server won't modify DNS records when assigning IP addresses.
recursion yes;	This enables the server to perform recursive queries. In recursion mode, if a client asks for a domain name that the server isn't authoritative for, BIND9 will resolve the query by querying other DNS servers on the Internet. This is common in caching DNS resolvers that serve internal clients.
listen-on {192.168.56.118; };	This directive tells BIND9 to only listen for DNS queries on the IP address 192.168.56.118. It restricts the server to handle queries only from the network interface associated with that IP, improving security by not exposing the service on all available interfaces.
allow-transfer {none;};	Zone transfers allow a secondary DNS server to replicate the data from a primary server. Setting `allow-transfer` to `{ none; }` disables all zone transfers. This is a security measure to prevent unauthorized copying of DNS zone data.
forwarders {8.8.8.8; 8.8.4.4};	This block defines external DNS servers (in this case, Google Public DNS servers) that BIND9 will use to forward queries it cannot resolve locally. When a query is received that the server isn't authoritative for and recursion is enabled, BIND9 forwards the query to these servers, improving response times and reliability.

The option table describes the purpose of each BIND directive. In particular, allow-transfer none prevents other DNS servers from copying the zone, while forwarders are used for names outside the local zone.

After that, in /etc/bind/named.conf.local file, configure the DNS zones to specify the zone name, zone type and the DNS record location for forward and reverse zone:

```
sang@22080031:~$ sudo nano named.conf.local
```

The named.conf.local file is opened to register the lab's authoritative zones. BIND uses these declarations to locate each zone's record file.



The forward zone example.com points to db.example.com, while the reverse zone for 192.168.56.0/24 points to db.56.168.192. Both zones are declared as primary zones on this server.

DNS Zone Configuration Table

Configuration	Description
zone "example.com" IN **or** **zone "56.168.192.in-addr.arpa" IN**	- **zone "example.com" IN { ... };** Declares a forward lookup zone for the domain `example.com`, handling the resolution of domain names to IP addresses. - **zone "56.168.192.in-addr.arpa" IN { ... };** Defines a reverse lookup zone for the IP address range `192.168.56.0/24`, facilitating the mapping of IP addresses back to domain names.
type master;	**type master;** Indicates that this server is the authoritative master for the `example.com` zone, maintaining the primary copy of the zone data. **Characteristics:** - **Authoritative Source:** Holds the definitive DNS records for the domain. - **Editable:** Administrators can directly add, modify, or delete DNS records within this zone. - **Zone Transfers:** Serves as the source for distributing DNS data to slave servers through a process called zone transfer.
file "/etc/bind/db.example.com" **or** **file "/etc/bind/db.56.168.192"**	- **Zone Files:** The specified files (`/etc/bind/db.example.com` and `/etc/bind/db.56.168.192`) contain the actual DNS records. For the forward zone, this includes A (Address) records mapping domain names to IP addresses. For the reverse zone, PTR (Pointer) records map IP addresses to domain names. - **Reverse Zone Naming:** The reverse zone's name (`56.168.192.in-addr.arpa`) corresponds to the `192.168.56.0/24` subnet. This naming convention is standard for reverse DNS lookups, where the octets of the subnet

The zone table explains how forward records map names to addresses and reverse records map addresses back to names. The file paths identify where BIND will read those records.

To properly configure DNS records, use the db.local file as a correct format template to configure db.example.com by copying the template to db.example.com file:

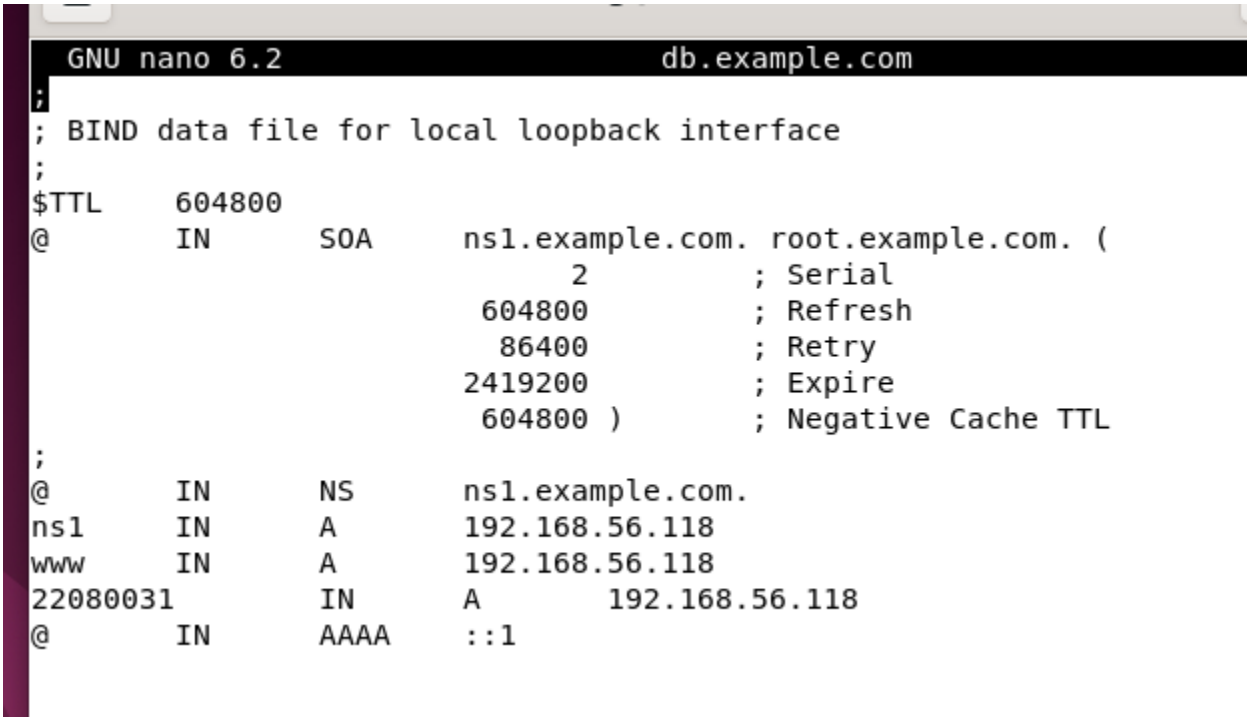
```
sang@22080031:/etc/bind$ sudo cp db.local db.example.com
```

The default db.local file is copied to create the forward-zone file. Using the supplied template provides the required DNS record structure before the lab-specific records are edited.

Then edit the DNS record in db.example.com:

```
sang@22080031:/etc/bind$ sudo nano db.example.com
```

The new db.example.com file is opened for editing. Its contents will define the authoritative records for example.com rather than relying on the default localhost template.



```
  GNU nano 6.2                    db.example.com
;
; BIND data file for local loopback interface
;
$TTL    604800
@       IN      SOA     ns1.example.com. root.example.com. (
                              2         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      ns1.example.com.
ns1     IN      A       192.168.56.118
www     IN      A       192.168.56.118
22080031        IN      A       192.168.56.118
@       IN      AAAA    ::1
```

The forward-zone screenshot shows the SOA and NS records plus A records for ns1, www, and the student-named host at 192.168.56.118. These A records let clients resolve those names to the server.

DNS Record Configuration Table

Configuration	Description
$TTL 604800	Sets the default Time-To-Live (TTL) to **7 days** (604800 seconds), controlling how long DNS records are cached.
@ IN SOA ns1.example.com. root.example.com. (...)	Defines the **Start of Authority (SOA)** record, specifying the primary name server and administrative email.
Serial: 2	Version number for the zone file, incremented when changes are made.
Refresh: 604800	Time (**7 days**) before secondary servers check for updates.
Retry: 86400	Time (**1 day**) before retrying a failed refresh.
Expire: 2419200	Time (**28 days**) before secondary servers stop using the zone file.
Negative Cache TTL: 604800	Time (**7 days**) to cache failed DNS lookups.
@ IN NS ns1.example.com.	Specifies **ns1.example.com** as the authoritative name server.
ns1 IN A 192.168.56.118	Maps **ns1.example.com** to IPv4 address **192.168.56.118**.
www IN A 192.168.56.118	Maps www.example.com to IPv4 address **192.168.56.118**.
22080031 IN A 192.168.56.118	Maps **22080031.example.com** to IPv4 address **192.168.56.118**.
@ IN AAAA ::1	Assigns **IPv6 loopback address ::1** to **example.com**.

The record table expands the SOA timers, nameserver entry, address records, and the IPv6 localhost entry. These values control zone metadata and the names that clients can look up.

Then the same configuration will be done on reverse DNS lookup in /etc/bind/db.127 file:

```
sang@22080031:/etc/bind$ sudo cp db.127 db.56.168.192
```

The reverse-zone template is copied and opened as db.56.168.192. This creates the file that answers reverse lookups for addresses in the 192.168.56 subnet.

```
sang@22080031:/etc/bind$ sudo nano db.56.168.192
```

The reverse-zone file is opened for editing after the copy. Its PTR records must use only the last octet of an address because the other octets are represented by the zone name.

```
 GNU nano 6.2                          db.56.168.192
;
; BIND reverse data file for local loopback interface
;
$TTL    604800
@       IN      SOA     ns1.example.com. root.example.com. (
                              1         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      ns1.example.com.
118     IN      PTR     ns1.example.com.
118     IN      PTR     www.example.com.
```

The reverse-zone screenshot assigns PTR records for address .118 to host names in example.com. A PTR query for 192.168.56.118 can therefore return a host name.

After all the setup, using **checkconf** and **in-add.arpa** to check the syntax error in every configuration file:

```
sang@22080031:/etc/bind$ sudo named-checkconf
sang@22080031:/etc/bind$ named-checkconf
sang@22080031:/etc/bind$ named-checkzone example.com db.example.com
zone example.com/IN: loaded serial 2
OK
sang@22080031:/etc/bind$ named-checkzone 56.168.192.in-addr.arpa db.56.168.192
zone 56.168.192.in-addr.arpa/IN: loaded serial 1
OK
sang@22080031:/etc/bind$
```

The named-checkconf and named-checkzone commands report that the configuration and both zones load successfully. These syntax checks catch malformed DNS files before client testing.

Task 3: Run the DNS server

Restart the DNS service and check its status.

```
sang@22080031:/etc/bind$ sudo service bind9 restart
sang@22080031:/etc/bind$ sudo service bind9 status
● named.service - BIND Domain Name Server
     Loaded: loaded (/lib/systemd/system/named.service; enabled; vendor preset:>
     Active: active (running) since Thu 2025-03-06 18:30:50 UTC; 9s ago
       Docs: man:named(8)
    Process: 5244 ExecStart=/usr/sbin/named $OPTIONS (code=exited, status=0/SUC>
   Main PID: 5245 (named)
      Tasks: 6 (limit: 4555)
     Memory: 5.8M
        CPU: 19ms
     CGroup: /system.slice/named.service
             └─5245 /usr/sbin/named -u bind

Mar 06 18:30:50 22080031 named[5245]: network unreachable resolving './NS/IN': >
Mar 06 18:30:50 22080031 named[5245]: network unreachable resolving './NS/IN': >
Mar 06 18:30:50 22080031 named[5245]: network unreachable resolving './NS/IN': >
Mar 06 18:30:50 22080031 named[5245]: network unreachable resolving './NS/IN': >
Mar 06 18:30:50 22080031 named[5245]: network unreachable resolving './NS/IN': >
Mar 06 18:30:50 22080031 named[5245]: network unreachable resolving './NS/IN': >
```

The BIND service is restarted and its status is shown as active. Restarting loads the newly edited zone files into the running DNS server.

Task 4: Verify DNS

rm /etc/resolv.conf to remove outdated or conflicting configurations:

```
sang@22080031:/etc/bind$ sudo rm /etc/resolv.conf
```

The old /etc/resolv.conf link is removed before changing the server's resolver settings. This is a lab-specific change; on a managed Ubuntu system, resolver configuration should normally be made through the active network service so it survives updates.

Then, create symbolic link to ensures that /etc/resolv.conf always reflects the latest system DNS settings:

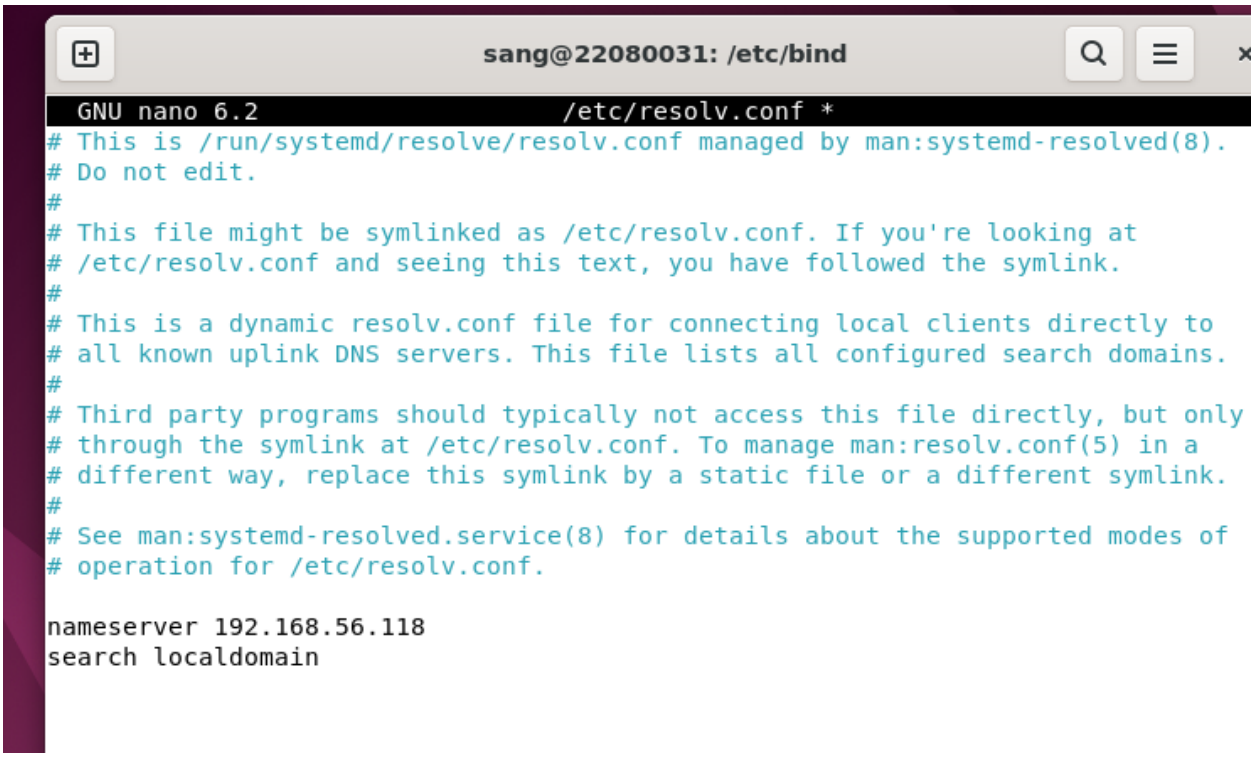
```
sang@22080031:/etc/bind$ sudo ln -sf /run/systemd/resolve/resolv.conf /etc/resol
v.conf
```

The screenshot creates a link from /etc/resolv.conf to systemd-resolved's resolver file. That link makes applications read the DNS settings from the systemd-resolved location.

Edit the resolv.conf file defines how the system resolves domain names into IP addresses:

```
sang@22080031:/etc/bind$ sudo nano /etc/resolv.conf
```

The resolver file is opened for editing. This step directs the server's own DNS queries to the local BIND instance rather than changing the DNS records BIND serves.



The file lists 192.168.56.118 as the nameserver. With that setting, local applications send DNS queries to the BIND service on the host-only interface.

DNS Resolver Configuration Table

Configuration	Description
nameserver 192.168.56.118;	Specifies **192.168.56.118** as the primary **DNS server**. Your system will use this IP address to resolve domain names to IP addresses.
search localdomain	Sets the **default domain name** for unqualified hostnames. If you type `server1`, the system will try `server1.localdomain` first before querying the DNS server.

The resolver table explains the nameserver and search-domain entries. The search domain applies only to short, unqualified names; the following test uses a fully qualified name.

Then, using nslookup to verify the resolve the domain name:

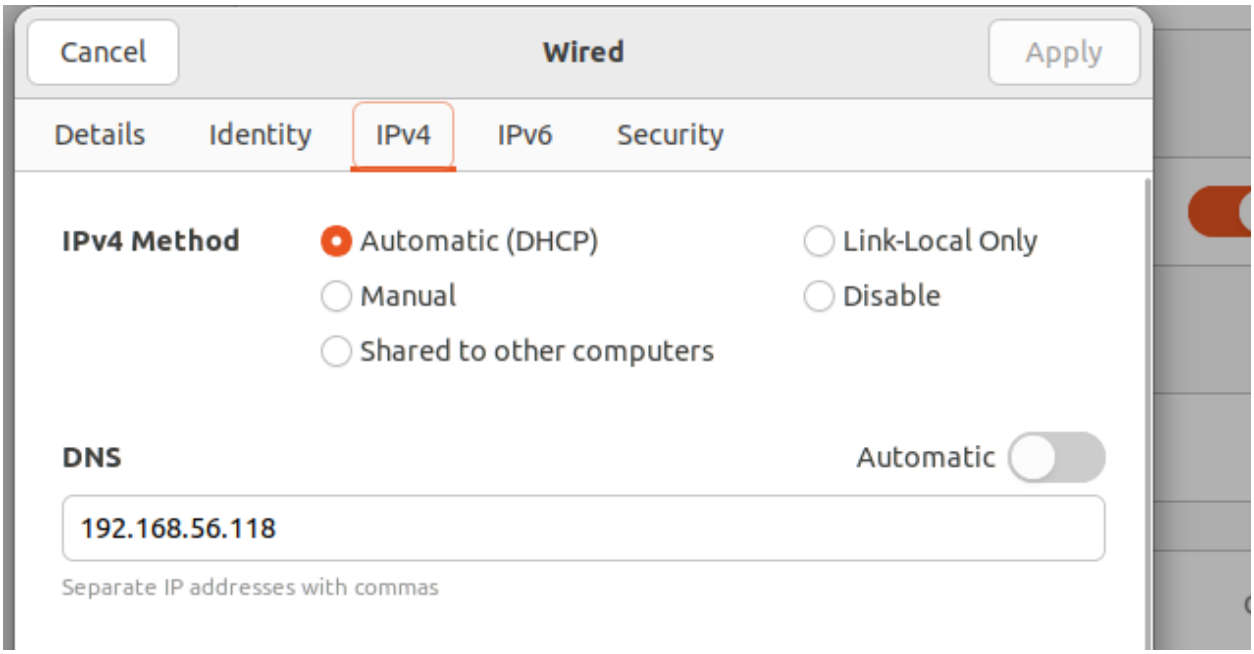
```
sang@22080031:/etc/bind$ nslookup www.example.com
Server:         192.168.56.118
Address:        192.168.56.118#53

Name:   www.example.com
Address: 192.168.56.118

sang@22080031:/etc/bind$ ping www.example.com
PING www.example.com (192.168.56.118) 56(84) bytes of data.
64 bytes from 22080031.example.com (192.168.56.118): icmp_seq=1 ttl=64 time=0.00
9 ms
64 bytes from 22080031.example.com (192.168.56.118): icmp_seq=2 ttl=64 time=0.03
1 ms
64 bytes from 22080031.example.com (192.168.56.118): icmp_seq=3 ttl=64 time=0.02
9 ms
```

The server's nslookup result resolves www.example.com to 192.168.56.118, and the ping uses that address. This confirms the forward zone answers the test name on the server.

Go to the guest machine and type the DNS server IP in the setting



The client network settings select 192.168.56.118 as the DNS server while retaining automatic IPv4 addressing. This separates address assignment by DHCP from name resolution through BIND.

Then configure the resolv.conf file:

```
(base) avocado@avocado-VirtualBox:~/Desktop$ sudo rm /etc/resolv.conf
(base) avocado@avocado-VirtualBox:~/Desktop$ sudo ln -s /run/systemd/resolve/res
olv.conf /etc/resolv.conf
(base) avocado@avocado-VirtualBox:~/Desktop$ sudo nano /etc/resolv.conf
(base) avocado@avocado-VirtualBox:~/Desktop$ sudo systemctl restart NetworkManag
er
```

The client removes and recreates /etc/resolv.conf, then restarts NetworkManager. This is the documented lab procedure; persistent DNS settings should be configured through the client's network manager.

```
  GNU nano 6.2                        /etc/resolv.conf
# This is /run/systemd/resolve/resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients directly to
# all known uplink DNS servers. This file lists all configured search domains.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 192.168.56.118
nameserver 1.1.1.1
search lan
```

The client's resolver file lists the server at 192.168.56.118, with another resolver as a fallback. The local server must be queried for the private example.com zone.

Finally, using nslookup to verify the resolve the domain name on the guest:

```
(base) avocado@avocado-VirtualBox:~/Desktop$ nslookup www.example.com
Server:         192.168.56.118
Address:        192.168.56.118#53

Name:   www.example.com
Address: 192.168.56.118
```

The final client nslookup resolves www.example.com through 192.168.56.118 to the same address. This is the end-to-end evidence that the client can reach the new DNS service.